Google Summer of Code

GSoC 2026 Application

Open Responses Protocol Support &
Generative UI Rendering Engine
for API Dash

Shridhar Panigrahi

sridharpanigrahi2006@gmail.com
github.com/sridhar-panigrahi
linkedin.com/in/shridhar-panigrahi-8bb176362

Polaris School of Technology, Bangalore
B.Tech Computer Science (AI & ML) | Expected 2029
IST (UTC+5:30)

Why I Want to Build This

Every tool I use to test AI APIs shows agentic responses as a flat `output[]` array. Reasoning, tool calls, results, final message — all mixed together at the same indentation level. You have to match `call_id` values by hand and scroll past reasoning blocks just to find what the model actually said. I checked Postman, Insomnia, and Thunder Client. All three are the same.

The Open Responses spec and A2UI spec both define exactly how this data is structured. That structure exists. No tool renders it. That's the kind of gap I notice and want to fix.

I built the PoC before writing this proposal — not to have something to show, but because I was debugging my own project and needed the tool. Read the codebase for two weeks first: how `_resolveViewOptions` decides which viewer to use, how the genai package abstracts providers, how `ResponseBodyView` routes formats. Made one wrong call early on — put A2UI detection in `ResponseBodySuccess` instead of `ResponseBody._resolveViewOptions` where all format decisions belong. Caught it and moved it. PR #1358 is the result: 1,400+ lines, 39 passing tests. I still use it.

What I want maintainers to know

- **I picked this over other GSoC ideas because I was already building it.** That matters when the work gets hard.
- **I read the code before I write code.** Every API Dash PR came after reading the relevant parts of the codebase first. I don't guess at architecture.
- **11 merged PRs across Python, Java, Go, Rust, and Dart.** I can read unfamiliar code quickly and write fixes that fit the project's existing style.
- **I'm already in the community** — weekly calls, Discord, the Idea 5 discussion thread. My PoC is the only one with a working streaming parser and unit tests.

Name and Contact Information

Name:	Shridhar Panigrahi
Email:	sridharpanigrahi2006@gmail.com
GitHub:	sridhar-panigrahi
LinkedIn:	shridhar-panigrahi-8bb176362
Time zone:	IST (UTC+5:30)
University:	Polaris School of Technology, Bangalore
Program:	B.Tech CSE (AI & ML)
Graduation:	Expected 2029

Motivation & Past Experience

---

1. Have you worked on or contributed to a FOSS project before?

Yes. 11 merged PRs across 5 projects (Python, Java, Go, Rust, Dart). Full details in the Contributions section below.

- **astropy** (Python) – 3 merged PRs: f-string bug, silent data corruption in `FITS_rec`, table index corruption.
- **Hyperledger Besu** (Java) – 4 merged PRs: missing return, wrong hash, TOCTOU race, unsigned underflow.
- **Hyperledger Fabric** (Go) – 3 merged PRs: gossip protocol error handling, iterator/resource leaks.
- **LFDT-Lockness/generic-ec** (Rust) – 1 merged PR: secp384r1 (NIST P-384) curve support.

API Dash (4 PRs):

- PR #1279 – Input validation. Learned how Riverpod state flows through the app.
- PR #1290 – Custom LLM providers. Gave me a working understanding of the genai package.
- PR #1321 – Idea doc for Idea 5: Open Responses & Generative UI.
- PR #1358 – Working PoC: 1,100 lines, 7 new files. Demos: Open Responses Viewer · A2UI Renderer.

2. What is your one project/achievement that you are most proud of? Why?

The PoC for this project (PR #1358). I spent about two weeks reading the codebase first – how `ResponseBodyView` works, how `ModelProvider` abstracts APIs, how Previewer routes to widgets. Then I wrote the sealed classes, auto-detection, structured viewer, and A2UI renderer.

Figuring out where to put things was harder than writing the code. I put the Open Responses models in `packages/genai` since other packages might need them. I put A2UI detection in `ResponseBody._resolveViewOptions()` since that's where format detection already happens. I got that wrong the first time – I put it in `ResponseBodySuccess`, which worked but was the wrong layer.

3. What kind of problems or challenges motivate you the most?

I like fixing things that are clearly broken or missing. For the astropy `FITS_rec` bug, I noticed negative indices weren't handled in slicing, I wrote a test, I fixed it. For the Besu TOCTOU race, I figured out two requests could hit the same block lookup at the same time. I find these by reading the code and thinking about what could go wrong.

Same with this project. AI API responses have structure (reasoning, tool calls, results, messages) but every tool just shows raw JSON. I checked Postman, Insomnia, Thunder Client – all the same. The Open Responses protocol and A2UI spec define clear rules for this structure, and they map well onto Flutter widgets. So I built a PoC to test it.

4. Will you be working on GSoC full-time?

Yes. Polaris has a long summer break that lines up with the GSoC coding period – no exams or classes during that time. I can commit approximately 3 hours a day (21 hours/week) during the coding period – mornings for focused coding, afternoons for reading and testing, evenings free for async discussions with mentors across time zones. My college schedule has no exams or classes during the GSoC coding period.

5. Do you mind regularly syncing up with the project mentors?

No. I already join the weekly GSoC calls and participate in the Idea 5 discussion thread. I prefer getting feedback early rather than finding out something is wrong after days of work. Available on Discord, email, or video.

6. What interests you the most about API Dash?

The architecture. It handles 60+ response types through one pipeline, and the parts I needed were already there – `ModelProvider`, `outputFormatter`, Previewer routing. The PoC worked without needing new plumbing, which means the codebase can handle a feature this size.

7. Can you mention some areas where the project can be improved?

- **AI response visualization:** Every other response type gets a proper viewer, but AI responses are still raw JSON. Structured card views for Open Responses format would close that gap.
- **Streaming experience:** Right now a streaming AI request shows raw SSE events in the body pane. A stream-aware viewer that builds cards progressively would make debugging much easier.
- **Multi-turn conversation view:** When you're chaining requests with `previous_response_id`, you want to see the full conversation, not just one request at a time. A timeline view for linked requests.
- **Agent UI preview:** Specs like A2UI let agents return UI component descriptors. No tool renders these yet. You'd see the actual rendered UI inside the API client instead of the raw JSON.

8. Have you interacted and helped the API Dash community?

Yes. I've been in the Idea 5 discussion thread since early on, sharing architecture decisions and getting feedback before writing the PoC. I attend the weekly GSoC calls and I'm active in the `#gsoc-foss-apidash` Discord channel. 4 PRs submitted – each one taught me a different part of the codebase (details in Q1 above).

Project Proposal Information

---

1. Proposal Title

Open Responses Protocol Support & Generative UI Rendering Engine for API Dash

2. Abstract

I've been testing AI APIs with Postman, Insomnia, and Thunder Client. All three show agentic responses (reasoning + tool calls + results + message) as raw JSON. I have to match `call_id` values manually, scroll past reasoning blocks, and figure out the execution flow from a flat array.

I built a working proof-of-concept in API Dash that solves this. It's open as **PR #1358** (∼1,400+ lines, 9 new files, 11 modified, **39 passing tests**). Video demos: **Open Responses Viewer** · **A2UI Renderer**. Idea doc: **PR #1321**.

It adds support for the Open Responses protocol and A2UI component rendering:

- Auto-detects Open Responses format and switches to a structured card view. Reasoning traces become collapsible cards, tool calls get matched to their results by `call_id`, web and file search calls get their own cards, messages render as markdown.
- Parses SSE streaming responses via `OpenResponsesStreamParser` – the same structured cards appear for both streaming and non-streaming responses.
- Renders A2UI component descriptors as real interactive Flutter widgets instead of raw JSON. Detects `surfaceTitle` and shows it as a header.
- **39 unit tests** (22 for Open Responses model/parser, 17 for A2UIParser) in `packages/genai`, runnable without Flutter.

GSoC adds: live progressive streaming (cards fill in character by character), conversation chaining via `previous_response_id`, DashBot integration, and production-grade test coverage.

3. Detailed Description

The Problem (and Why It Matters)

When you test an AI API that uses tool calling – say, asking a model to look up the weather – the response looks something like this:

```json
{
  "id": "resp_abc123",
  "object": "response",
  "status": "completed",
  "output": [
    {
      "type": "reasoning",
      "id": "rs_001",
      "summary": [{"type": "summary_text",
        "text": "Deciding to call weather tool..."}]
    },
    {
      "type": "function_call",
      "id": "fc_001",
      "name": "get_weather",
      "call_id": "call_xyz",
      "arguments": "{\"city\": \"Tokyo\"}"
    },
    {
      "type": "function_call_output",
      "call_id": "call_xyz",
      "output": "{\"temp\": 22}"
    },
    {
      "type": "message",
      "id": "msg_001",
      "content": [{"type": "output_text",
        "text": "It's 22C in Tokyo."}]
    }
  ],
  "usage": {"input_tokens": 150,
    "output_tokens": 45, "total_tokens": 195}
}
```

Every API testing tool I've tried renders this as a raw JSON tree. Current state vs what I'm building:

TODAY

Postman • Insomnia • Thunder Client

Raw JSON tree. One flat `output[]` array.

- Which `function_call_output` matches which call? Compare `call_id`s manually.
- Where is the actual answer? Scroll past reasoning blocks to find it.
- What order did things happen? Reconstruct from a flat array.
- SSE stream? Raw event text dumped in the body pane.
- A2UI components? Unrendered JSON blobs.

AFTER

API Dash with this project

Structured card view. Each output item is its own widget.

- Tool calls & results auto-grouped by `call_id` — matched in one expandable card.
- Message rendered as formatted markdown, inline images included.
- Reasoning collapsed by default, full chain-of-thought one click away.
- SSE stream? Cards appear live, text fills in character by character.
- A2UI components? Real interactive Flutter widgets — tap, type, interact.

You have to mentally track which `function_call_output` matches which `function_call` by comparing `call_id` values, scroll past reasoning blocks to find the actual message, and reconstruct the execution flow from a flat array.

The Open Responses protocol is being adopted across providers – OpenAI has shipped it. Google's A2UI spec lets agents return UI component descriptors (buttons, forms, tables, charts) instead of

just text, and the client renders them as interactive widgets. No tool does this today.

API Dash already handles 60+ response types through one pipeline, has a genai package with provider abstractions, and has SDUI infrastructure. I didn't need to build new plumbing for the PoC.

What I've Already Built (Proof of Concept)

I built a working PoC before writing this proposal. It's open as **PR #1358**.

What I built: 9 new files, 11 modified, ~1,400+ lines of Dart, 39 passing unit tests. Video demos: Open Responses Viewer · A2UI Renderer.

1. Open Responses Data Models (`packages/genai/lib/models/open_responses.dart` – 269 lines)

The Open Responses `output[]` array mixes messages, function calls, results, and reasoning together. I used Dart sealed classes so I don't have to do manual `type` string checks – the type system handles dispatch:

```dart
// using sealed here so dart forces exhaustive matching
// if i add a new output type later, the compiler will remind me
sealed class OutputItem {
  const OutputItem();

  String get id;
  String get type;
  String get status;

  // just switch on 'type' and return the right subclass
  factory OutputItem.fromJson(Map<String, dynamic> json) {
    return switch (json['type'] as String? ?? '') {
      'message' => MessageOutputItem.fromJson(json),
      'function_call' => FunctionCallOutputItem.fromJson(json),
      'function_call_output' => FunctionCallResultItem.fromJson(json),
      'reasoning' => ReasoningOutputItem.fromJson(json),
      _ => UnknownOutputItem.fromJson(json), // safe fallback
    };
  }
}
```

2. Structured Response Viewer (`lib/widgets/open_responses_viewer.dart` – 537 lines)

The viewer widget. Each output item type gets its own card:

- `_MessageCard` – Role-labeled chat bubbles with GitHub-flavored markdown rendering via `MarkdownBody`, user messages right-aligned with avatar, assistant messages left-aligned.
- `_ReasoningCard` – Collapsible reasoning sections showing summary by default, expandable to show full chain-of-thought.
- `_ToolCallGroup` – Tool calls grouped with their results by `call_id`. Shows function name, status chip (`completed`/`in_progress`/`failed`), pretty-printed JSON arguments, and matched result – all in one expandable card.
- `_WebSearchCard` – Compact card for `web_search_call` items showing a search icon and status chip.
- `_FileSearchCard` – Card for `file_search_call` items, listing each query with a subdirectory arrow and status chip.

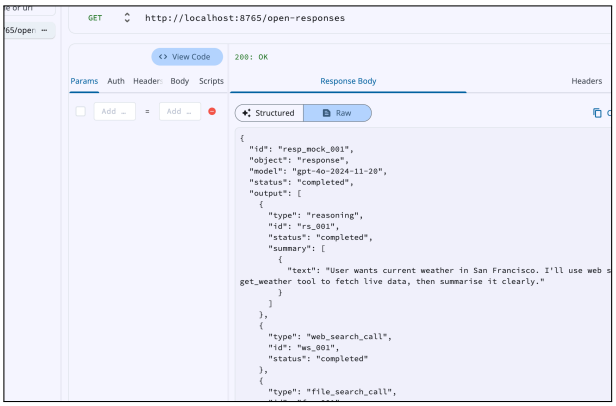
- `_CopyIconButton` – Copy-to-clipboard button on all cards for quick data extraction.
- `_UsageBar` – Token usage statistics (input/output/total) displayed at the bottom.
- **Refusal handling** – `RefusalPart` content renders with error styling and red-tinted background.

All cards follow Material 3, work in dark and light mode. The core method is `_buildGroupedItems()`, which pairs tool calls with their results before rendering:

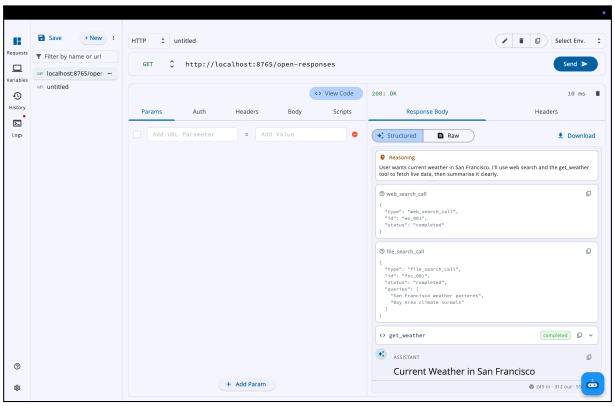
```
// this is where the magic happens -- matching fn calls to results
List<Widget> _buildGroupedItems(BuildContext context) {
  // collect all results first so i can look them up by call_id
  final resultMap = <String, FunctionCallResultItem>{};
  for (final item in result.output) {
    if (item is FunctionCallResultItem) {
      resultMap[item.callId] = item;
    }
  }

  final widgets = <Widget>[];
  for (final item in result.output) {
    if (item is FunctionCallResultItem) continue; // skip, already in map

    if (item is FunctionCallOutputItem) {
      // find the matching result for this call
      final matchedResult = resultMap[item.callId];
      widgets.add(
        _ToolCallGroup(call: item, result: matchedResult),
      );
    } else {
      // rest of the items get their own card widget
      widgets.add(switch (item) {
        MessageOutputItem() => _MessageCard(item: item),
        ReasoningOutputItem() => _ReasoningCard(item: item),
        UnknownOutputItem() => _UnknownCard(item: item),
        _ => const SizedBox.shrink(),
      });
    }
  }
  return widgets;
}
```



Raw Open Responses JSON — how every tool shows it today



PoC: structured card view – reasoning, web search, file search, tool calls grouped by `call_id`*, and markdown message. Works for both streaming and non-streaming responses.*

3. A2UI Component Renderer (`lib/widgets/a2ui_renderer.dart` – 349 lines)

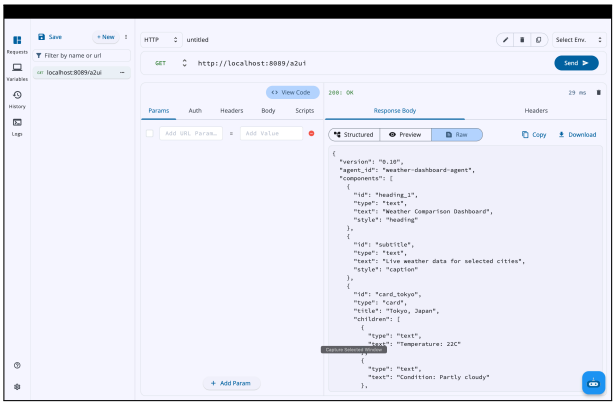
A2UI sends a JSON list of component descriptors and the client renders UI from that. I built a registry that maps component type strings to Flutter widget builders, and a parser that detects A2UI payloads in JSONL format. I made the renderer stateful so interactive components (checkboxes, text fields) keep their own state. It currently handles **15 component types**:

- **Layout:** Card, Row, Column, List, Tabs
- **Display:** Text (with h1/h2/h3/caption variants), Image, Icon, Divider
- **Input:** Button (primary/outlined/text variants), TextField, Checkbox, Switch
- **Feedback:** Progress (determinate/indeterminate), Chip
- **Data Binding:** JSON Pointer path resolution from the data model (e.g., `/user/name`)

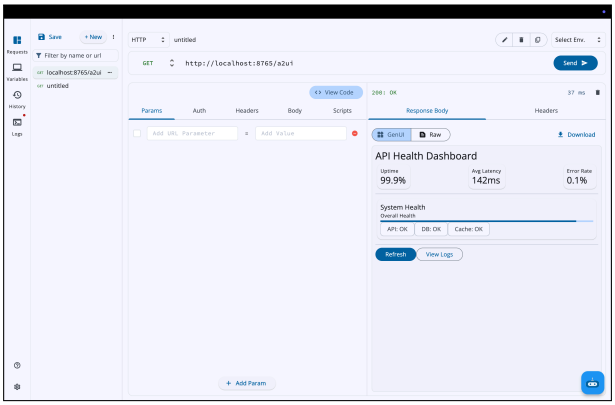
TextFields, Checkboxes, and Switches keep state locally. Buttons show a snackbar with the action name. Unknown component types fall back to an error card with the type name instead of crashing. The routing method:

```dart
// takes a component id, looks it up, and builds the right widget
// card/row/column call this recursively for their children
Widget _renderComponent(BuildContext context, String id) {
  final node = widget.components[id];
  if (node is! Map<String, dynamic>) return const SizedBox.shrink();

  // big switch -- one builder per component type
  return switch (node['component'] as String? ?? '') {
    'Text' => _buildText(context, node),
    'Button' => _buildButton(context, node),
    'Card' => _buildCard(context, node),
    'Row' => _buildRow(context, node),
    'Column' => _buildColumn(context, node),
    'Image' => _buildImage(node),
    'Icon' => _buildIcon(node),
    'Divider' => const Divider(),
    'List' => _buildList(context, node),
    'Tabs' => _buildTabs(context, node),
    'TextField' => _buildTextField(context, node),  // these three are stateful
    'Checkbox' => _buildCheckbox(context, node),
    'Switch' => _buildSwitch(context, node),
    'Progress' => _buildProgress(context, node),
    'Chip' => _buildChip(context, node),
    final t => _A2UIError(message: 'Unknown: $t'), // won't crash on new types
  };
}
```



A2UI component descriptors shown as raw JSON — no tool renders this today



PoC: same payload rendered as a live interactive dashboard – stat cards with data binding, progress bar, chips, and buttons. Full A2UI spec coverage is planned for GSoC.

4. Response Pipeline Integration

The existing pipeline has a `_resolveViewOptions()` function that picks which viewer to use. I added two new `ResponseBodyView` options and put the detection logic there:

- `structured` view – kicks in when the response has `object == "response"` and an `output[]` array. Routes to the Open Responses viewer.
- `structured + sse + raw` tabs (`kStructuredSSEBodyViewOptions`) – kicks in when the SSE output contains `response.output_item` events. The `OpenResponsesStreamParser` reassembles deltas into the same `OutputItem` list, so the structured viewer renders streaming and non-streaming responses identically.
- `genui` view – kicks in when the body contains `createSurface` or `updateComponents` JSONL

keys. Routes to the A2UI renderer.

- `ResponseBodySuccess` then just renders the right one based on which view option was resolved.

How I'll Build This – Technical Approach for GSoC

The PoC works on fixture data. Real responses have edge cases I haven't handled yet. Here's what I need to build during GSoC:

A. Streaming with Typed SSE Events

Open Responses streaming events are typed and target specific items by index. You get `response.output_item.added` when a new item starts, then a series of `response.output_text.delta` events that each say "append this text to item at index N", and finally `response.output_item.done` when that item is complete. So the parser needs to keep a running map of in-progress items and route each delta to the right card in the UI. The PoC viewer already renders each item type as its own card (as shown above), so the streaming layer just needs to feed items into that same widget tree incrementally instead of all at once.

B. Conversation Chaining (`previous_response_id`)

Open Responses supports continuing a conversation by passing the previous response's ID in the next request. Right now API Dash treats every request as standalone, so I need to change that:

- When we parse a response, pull out the `response.id` and store it on the request model.
- Add a field to the request editor so you can paste a `previous_response_id` to chain off a prior response.
- Build a timeline view that links chained requests so you can see the whole conversation flow in one place.
- Each turn in the chain should be expandable/collapsible so you're not overwhelmed if the conversation is long.

C. Extended A2UI Component Support

I have 15 component types in the PoC. That's enough for a demo, not enough for real use. During GSoC I need to cover enough of the A2UI spec that real Vertex AI agent responses just work:

- **More input components:** DatePicker, Slider, Radio, Dropdown (these are common in agent UIs but not in the PoC yet)
- **Data display:** Table with sorting and filtering, Charts via `fl_chart`
- **Navigation components:** Accordion, Stepper – needed for multi-step agent flows
- **Form state:** Right now state is per-component; I need to unify it so a form submission can read all field values at once
- **Streaming:** A2UI uses a flat ID-referenced model where you can add/update components incrementally. Components should render as they arrive, not wait for the whole payload

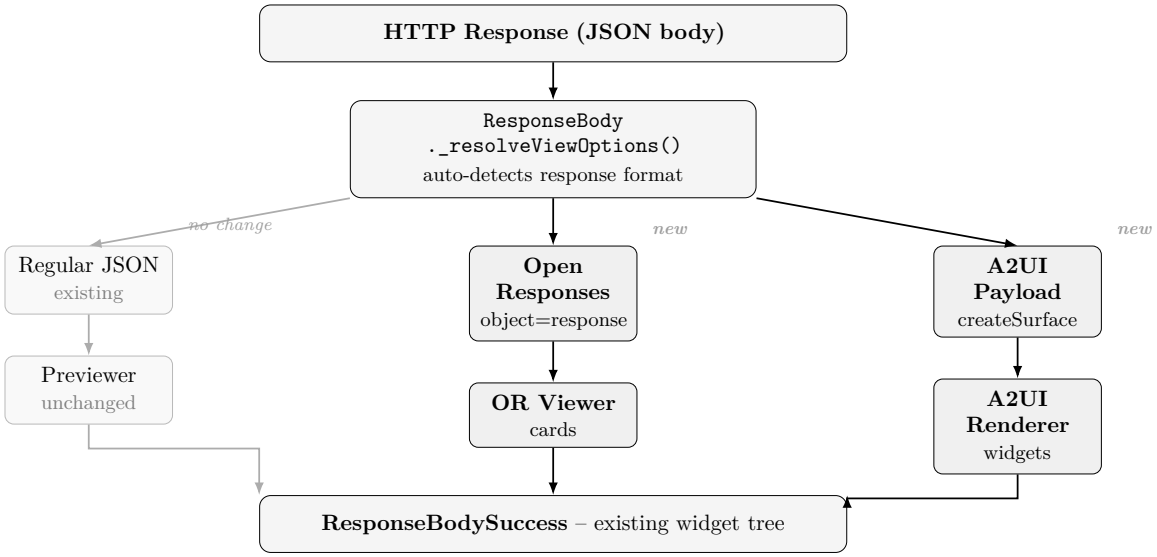
D. DashBot Integration

DashBot passes everything through `MarkdownBody` inside `ChatBubble`. Works for plain text, but Open Responses format shows up as raw JSON. Fix: check the response format before rendering:

- If it's Open Responses format, run it through the same `OpenResponsesResult.fromJson` pipeline and render the structured cards inline in the chat (reasoning collapsed by default, tool calls as expandable cards).

- If it contains A2UI components, render the interactive widgets right inside the chat bubble, so the agent's UI shows up where the conversation is.
- Plain text/markdown responses stay exactly as they are now.

Architecture Diagram



What the End Result Will Look Like

Scenario 1: Non-streaming Open Responses. You send a request. API Dash detects the format and switches to structured view. Reasoning card collapsed at top, tool call card with matched result below it, message as markdown at bottom. Token usage bar at the very end.

Scenario 2: Streaming. Cards appear live as SSE events arrive. Reasoning card fills in progressively, then a function call card shows up with arguments growing character by character, then the result slides in, then the message. No raw JSON fragments visible at any point.

Scenario 3: A2UI agent response. A Vertex AI agent returns component descriptors. Instead of JSON, you see rendered Flutter widgets: buttons you can tap, text fields you can type in, data tables with rows. You test the agent's UI without building a separate frontend.

Scenario 4: Multi-turn debugging. Three chained requests using `previous_response_id`. API Dash links them into a timeline. You see the full flow in one connected view instead of three separate tabs.

What Could Go Wrong and How I'd Handle It

Risk	Impact	Mitigation
Extending outputFormatter	High	Add a parallel `structuredOutputFormatter` path
A2UI spec changes	Medium	Pin to specific version; unknown types show raw JSON
Streaming takes longer	Medium	Non-streaming structure works first as fallback
Multi-turn chain state	Medium	Start with in-session track, persist later
Testing without API keys	Low	Already solved – PoC includes fixture payloads

Implementation Plan

Project Size: 90 hours over 12 weeks (~3 hours/day on coding days)

Community Bonding Period

- Attend weekly mentor sync-ups and get feedback on the PoC.
- Address any review comments on PR #1321 (idea doc) and the PoC PR.
- Finalize design decisions with mentors: how to extend the provider interface for structured output and the A2UI rendering approach.
- Set up test fixtures for all edge cases (malformed responses, partial data, unknown output types).

Phase 1 – Open Responses Production Parser & Structured Viewer (Weeks 1–3)

Goal: Turn the PoC into code that's ready to merge.

Week 1

- Finalize the `OpenResponsesResult` data models based on mentor feedback.
- Add handling for `output_image` and `output_file` content parts (PoC only handles `output_text` and refusal).
- Make format auto-detection reliable – zero false positives on non-AI responses.
- Write unit tests for all `fromJson` factories using fixture payloads.

Deliverable: Open Responses parser in `packages/genai` with full test coverage.

Week 2

- Polish the Structured Response Viewer widgets (reasoning, function call, message cards).
- Add markdown rendering for message text content (using existing `flutter_markdown` integration).
- Implement inline image rendering for `output_image` parts (using existing `Image.memory`).
- Add a download button for `output_file` parts.

Deliverable: Complete structured viewer rendering all Open Responses output types.

Week 3

- Wire the `OpenResponsesModel` provider into `kModelProvidersMap`.
- Implement `createRequest()` for the Responses API format (input + instructions).
- Implement non-streaming `outputFormatter` and `structuredOutputFormatter`.
- Integration tests: send requests via UI, verify auto-detection and structured rendering.

Deliverable: Working end-to-end flow – user sees structured cards instead of raw JSON.

Phase 2 – Streaming (Weeks 4–5)

Goal: Handle real-time streaming responses with progressive structured rendering.

Week 4

- Build the `OpenResponsesStreamState` machine for typed SSE events.

- Handle core event types:
  - `response.output_item.added`
  - `response.output_text.delta`
  - `response.function_call_arguments.delta`
  - `response.output_item.done`
- Wire stream state into `streamOutputFormatter` for the Open Responses provider.

Deliverable: Streaming parser that correctly routes typed deltas to output items.

Week 5

- Connect the stream state machine to the structured viewer for progressive rendering.
- Each output item card appears as `output_item.added` fires; text and arguments fill in as deltas arrive.
- Handle edge cases: out-of-order events, partial data, connection drops.
- Write tests with recorded SSE fixtures.
- Mentor review before midterm evaluation.

Deliverable: Live streaming structured view – cards build up in real-time.

Midterm Evaluation

Expected state: Full Open Responses support (parsing, structured viewing, streaming) working end-to-end.

Phase 3 – A2UI Component Renderer (Weeks 6–7)

Goal: Expand the A2UI renderer with more components and mixed-response support.

Week 6

- Expand the component registry to cover additional A2UI types beyond the PoC: Radio, Dropdown, Table, Accordion, Stepper.
- Implement data binding resolution for all input types.
- Fix form state management so it works across components, and handle unknown types without crashing.

Deliverable: A2UI renderer supporting **20+** component types with interactive state.

Week 7

- Build the A2UI detection and routing pipeline (extending `_resolveViewOptions()`).
- Add support for mixed responses (Open Responses output containing A2UI component descriptors).
- Write tests for component rendering with various payloads.
- Mentor review.

Deliverable: Tested A2UI rendering pipeline integrated into the response viewer.

Phase 4 – Conversation Chaining & DashBot Integration (Weeks 8–9)

Goal: Connect multi-turn agentic workflows and bring structured responses into DashBot.

Week 8

- Implement `previous_response_id` tracking in the request model.
- Build the conversation timeline view linking related requests.
- Allow expanding/collapsing individual turns in the chain.

Deliverable: Multi-turn conversation chaining with a connected timeline view.

Week 9

- Extend `ChatBubble` in DashBot to render structured Open Responses output inline.
- When DashBot receives a response with reasoning/tool calls, render them as cards within the chat.
- Add A2UI component rendering within chat bubbles for interactive agent responses.

Deliverable: DashBot with rich structured response rendering.

Phase 5 – Testing, Polish & Documentation (Weeks 10–12)

Goal: Test everything, fix bugs, write docs, get it ready to merge.

Week 10

- End-to-end testing with real API endpoints (OpenAI Responses API, compatible providers).
- Edge case handling: malformed responses, partial streams, mixed content types, very large responses.
- Performance profiling: ensure structured rendering does not add noticeable latency.

Deliverable: Implementation tested against real APIs and edge cases.

Week 11

- Fix all bugs identified during testing.
- Code cleanup and ensure all new code follows API Dash's existing patterns.
- Write developer documentation for extending the parser and A2UI component registry.

Deliverable: Clean, documented implementation ready for final review.

Week 12

- Address any last review comments from mentors.
- Write the final project report and submit final evaluation.

Deliverable: Code ready to merge, docs written.

Stretch Goals

If the core work finishes ahead of schedule:

- **Full A2UI Spec:** The PoC handles 15 component types. The rest of the spec includes event logging (see what interactions the agent's UI generates), progressive rendering via `updateComponents`, and component update-by-ID so live agent UIs can mutate in place. This turns the renderer from a preview tool into a debugging tool.
- **A2UI Live Playground:** Split-pane editor – A2UI JSON on the left, live rendered Flutter widgets on the right, updating as you type. Would make iterating on agent UIs much faster. No tool does this currently.

- **Visual Conversation Debugger:** Multi-turn agentic loops are hard to follow. A view showing the full conversation tree – collapsible branches, filter by output type, jump to any turn – with export as a shareable report.

Contributions to API Dash

PR	Description	Status
#1279	Input validation for empty API key and endpoint URL in AI Model Selector dialog	**Open**
#1290	Custom OpenAI-compatible LLM provider support (Groq, OpenRouter, Mistral, etc.)	**Open**
#1321	GSoC 2026 idea doc – Open Responses & Generative UI	**Merged**
#1358	PoC – Open Responses structured viewer (reasoning, web/file search, tool call grouping, markdown), SSE streaming parser, A2UI renderer with 15 components and data binding, 39 unit tests (demo 1, demo 2)	**Open**

Other Open-Source Contributions

Project	PR	Description	Status
astropy (Python)	#19403	Missing f-string prefix in `BaseAffineTransform`	**Merged**
	#19404	Silent data corruption in `FITS_rec` slice assignment	**Merged**
	#19450	Table index corruption on failed row assignment	**Merged**
Besu (Java)	#10020	Missing return in `serializeAndDedupOperation()`	**Merged**
	#10042	Wrong hash in `eth_estimateGas`	**Merged**
	#10051	TOCTOU race in `eth_getBlockByNumber`	**Merged**
	#10060	Unsigned underflow in `eth_feeHistory`	**Merged**
Fabric (Go)	#5422	Gossip protocol missing return after NACK	**Merged**
	#5424	Resource leak in `collItr`	**Merged**
	#5430	Iterator leak in `deleteDataMarkedForPurge`	**Merged**
generic-ec (Rust)	#59	secp384r1 (NIST P-384) curve support, 180 tests	**Merged**

Why Me

I am the person who already built the PoC. Not to have something to show for an application — because I was debugging my own AI project and I needed the tool. Here is what that actually required:

I started with PR #1279 (input validation) to understand how Riverpod state flows through the app before touching anything deeper. Then PR #1290 (custom LLM providers), which required reading the entire genai package: how `ModelProvider` abstracts backends, how `createRequest()` formats input, how `outputFormatter` pipes results back. I know that pipeline well enough to extend it,

not just call it. Then I built the PoC: sealed class hierarchy for the Open Responses output types, 7-card structured viewer, A2UI renderer with 15 components, SSE streaming parser, 39 unit tests. All before applying. PR #1358 is the result.

What the PoC already proves:

- The format auto-detection works: Open Responses JSON, Open Responses SSE stream, and A2UI JSONL each route to the right viewer with zero false positives on regular JSON responses.
- The sealed class dispatch is correct: all five output item types (`Message`, `Reasoning`, `FunctionCall`, `WebSearchCall`, `FileSearchCall`) each render as their own card type.
- The A2UI renderer handles interactive state: TextFields, Checkboxes, and Switches keep local state; unknown component types fall back gracefully without crashing.
- The streaming parser assembles a complete `OutputItem` list from SSE deltas — the same viewer renders streaming and non-streaming responses identically.
- The models live in `packages/genai` and are tested without Flutter. The test suite runs in 39ms.

The remaining GSoC work (progressive streaming, conversation chaining, DashBot integration, extended A2UI components) is extension, not invention. The foundation is already in the codebase and passing tests.

What will be hard:

- **Streaming (live progressive rendering)** – The PoC has `OpenResponsesStreamParser` which reassembles SSE deltas into a final `OutputItem` list and the structured viewer already renders it correctly. What's missing for GSoC is the *progressive* part – cards should fill in character by character as deltas arrive, not wait for the stream to complete. This means wiring the parser's incremental state into a Riverpod stream so each delta triggers a widget rebuild at the right card, not a full re-render.
- **Conversation chaining** – The current request model doesn't store response IDs. Need to extend it to persist `response.id` so the next request can reference it via `previous_response_id`. Raised this in the Idea 5 discussion thread and haven't got an answer yet on where to store it – community bonding decision.

Broader experience: 11 merged PRs across 5 open-source projects in Python, Java, Go, Rust, and Dart. Bugs fixed include silent data corruption (astropy #19404), race conditions (Besu #10051), and resource leaks (Fabric #5424).

By the end of summer, when I send an agentic API request in API Dash, I want to see structured cards, not raw JSON. The PoC proves the approach works. GSoC is how I take it the rest of the way.